Diffusion-Based Grasp Sampling

Project 2: a compact conditional sampler for TAMPanda

Mohammed Issa, Nils Bergau, Xuen

October 8, 2026

Table of Contents

1. Theory
2. Model implementation
3. Repository quick glance
4. Model training and denoising processes
5. Testing and results
6. Conclusion

1.1 - Diffusion in one slide

- A diffusion model learns to recover a clean sample from a noisy version of that sample, usually an image
- Our sample: a normalized 17-value grasp vector x_0
- Forward process: add noise at a randomly selected time step t

Forward noising

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I)$$

- $\bar{\alpha}_t = \prod_{s \le t} \alpha_s$ with $\alpha_t = 1 - \beta_t$ and β_t is the noise added at step t
- **Denoising step:** one model prediction plus one scheduler update from x_t toward x_{t-1}
- **Sampling:** the complete reverse chain from random noise to a grasp candidate

1.2 - Reverse update, DDIM, DDPM and the baseline

	DDIM	**DDPM**
Randomness	Initial noise; deterministic reverse	Initial and reverse-step noise
Role here	Deterministic comparison	Stochastic comparison
Final budget	100 updates	100 updates

Update rule

$$\hat{x}_0 = f_\theta(x_t, c, t), \qquad x_{t-1} = \text{SchedulerStep}(x_t, \hat{x}_0, t)$$

- The object condition c contains position, half-size and orientation
- DDPM samples a stochastic transition:

$$x_{t-1} = \mu_\theta(x_t, \hat{x}_0, t) + \sigma_t z, \qquad z \sim \mathcal{N}(0, I).$$

- DDIM is deterministic when $\eta = 0$; DDPM keeps reverse-step noise
- Both use the same trained denoiser; only the scheduler changes
- Baseline analytical TAMPanda `GraspPlanner` against DDIM/DDPM

2.1 - Model input and output

Clean grasp x_0: 17 values

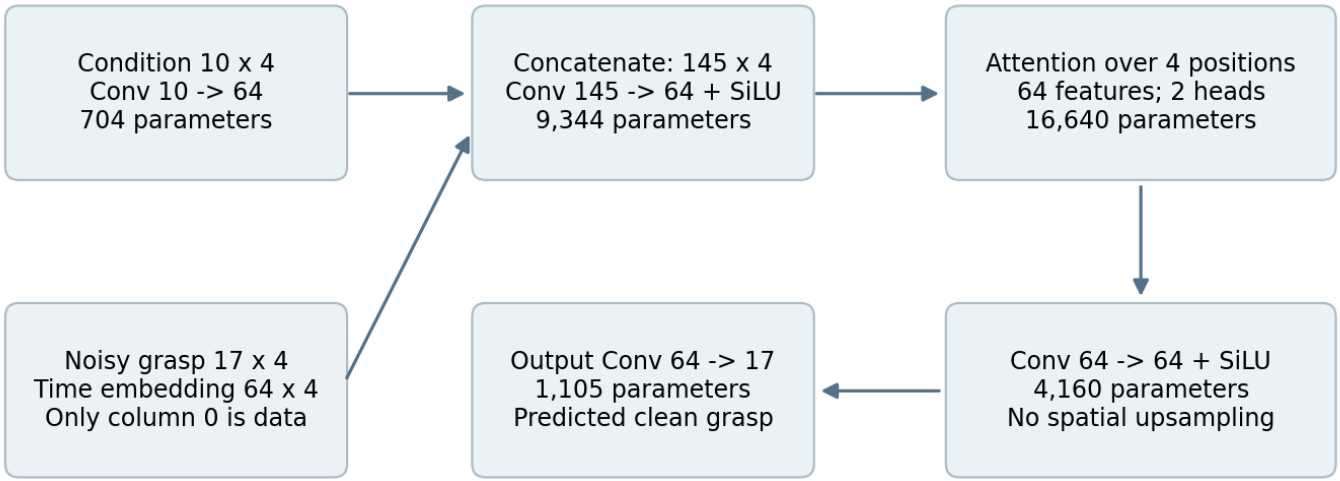
- 3 grasp-type values: top-down-y, top-down-x, front
- 4 quaternion values
- 3 approach-position values
- 3 grasp-position values
- 3 lift-position values
- 1 grasp score

Condition c: 10 values

- object position (x, y, z)
- object half-size (x, y, z)
- object orientation quaternion

Values are normalized using training-set mean and standard deviation, then denormalized before creating a native `GraspCandidate`

2.2 - The compact conditional denoiser



- Condition projection: $10 \rightarrow 64$ channels
- Sinusoidal time step embedding: 64 values
- Concatenate noisy grasp, condition and timestep: $17 + 64 + 64 = 145$ channels
- 1×1 projection to 64 hidden channels, followed by SiLU
- Two-head self-attention over the four tensor positions
- $64 \rightarrow 64$ projection, SiLU, then $64 \rightarrow 17$ output projection
- Shallow 1D U-Net-like denoiser

2.3 - Why these model parameters?

Choice	Value	Reason
Hidden width	64	Capacity for a 17-value vector while keeping the model small
Time step width	64	Matches the hidden scale for 1,000 sinusoidal noise levels
Attention heads	2	Two 32-dimensional heads give cross-feature interaction cheaply
Kernel size	1	Mixes channels without imposing an image-like spatial neighbourhood
Prediction target	x_0	Directly decodes to physical grasp values and uses the scheduler's `sample` mode
Clip sample	false	Avoids clipping normalized outputs before geometry and quaternion validation

These are practical engineering choices informed by the small dataset and the budget.

3.1 - How the model maps onto the repository

Repository file	Connection to the model and experiment
`data/processing.py`	Encodes the 17-value grasp, the 10-value condition, normalization, and candidate decoding
`model/unet1d.py`	Defines the custom conditional denoiser and retains the optional Hugging Face branch
`model/training_pytorch.py`	Adds noise, drops conditions for guidance training, computes masked x_0 loss, and saves checkpoints
`model/config.py`	Defines training timesteps, scene split, normalizer metadata and validation loss
`pddl/init.py`	Loads one frozen checkpoint and selects DDIM or DDPM plus the inference step count
`pddl/evaluate_diffusion.py`	Runs the reverse loop and combines conditional/unconditional predictions
`model/sampler.py`	Builds conditions, starts Gaussian noise, denormalizes output and returns batches
`pddl/checks.py`	Tests geometry, IK, lift and measured placement outcomes
`pddl/benchmark.py`	Creates paired scenes/goals and records candidate and task metrics

`condition` → `sampler` → `DDIM/DDPM` → `candidate` → `checks/benchmark`

4.1 - Training settings and the learning target

Data and training

- 2,000 scenes and 4,968 teacher grasp labels
- 1,700/300 scene split, seed 42
- batch size 32, 200 epochs
- AdamW, learning rate 10^{-4}
- gradient norm clipped to 1.0

Diffusion training

- $T = 1,000$ training noise levels
- random time step per example
- model predicts clean x_0
- 10% condition dropout for guidance
- masked loss ignores padded columns

- The analytical planner provides supervision, so this is teacher imitation rather than learning from human contact labels
- The clean grasp is a training label only. During generation the model receives noisy grasp, object condition and time step

4.2 - Denoising and sampling in the repository

Reverse chain

1. Start with $x_T \sim \mathcal{N}(0, I)$
2. For $t = T, \ldots, 0$, predict $\hat{x}_0 = f_\theta(x_t, c, t)$
3. Apply the DDIM or DDPM scheduler to obtain x_{t-1}
4. Reset padding columns to zero
5. Decode and validate the final grasp

Classifier-free guidance

$$f_{guided} = f_{uncond} + w(f_{cond} - f_{uncond})$$

- Final study: guidance $w = 1$, with 100 inference updates
- Earlier tests: guidance $w = 3$ and a matched step sweep
- A denoising step is one scheduler/model update. A sampling run is the complete chain that produces one candidate batch

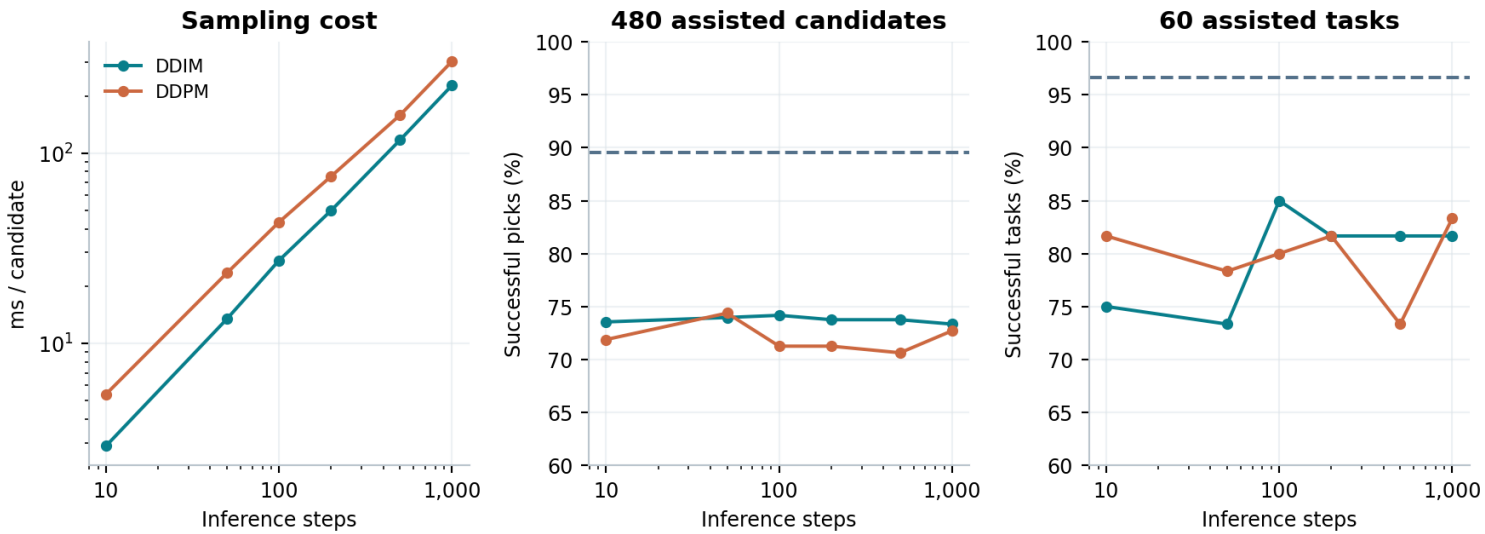
Why 100 updates?

5.1 - Testing strategy

Stage	What was tested	Why it matters
Tests 01-04	Setup, smoke run, sampler speed, type-/pose coverage	Find import, tensor and proposal problems early
Tests 05-11	DDIM/DDPM budgets 10-1,000, initially attachment-assisted	Compare step cost and sampler behaviour fairly
Tests 12-18	Native calls, placement trace, physical grasp and regression fixes	Separate adapter and executor faults from grasp quality
Tests 19-21	Corrected physical tasks with guidance 3	Establish the pre-intervention physical result
Tests 23-28	Guidance 1, 100 steps, physical tasks and isolated lifts	Final learned-method evaluation on three seed groups
Test 29	Six selected native demonstrations	Show end-to-end native integration, not a cohort estimate

- Physical lift success requires the cube centre to rise at least 25 mm
- Placement success requires final XY error $\leq$ 30 mm and Z error $\leq$ 25 mm
- Sampling time excludes IK, motion planning and execution

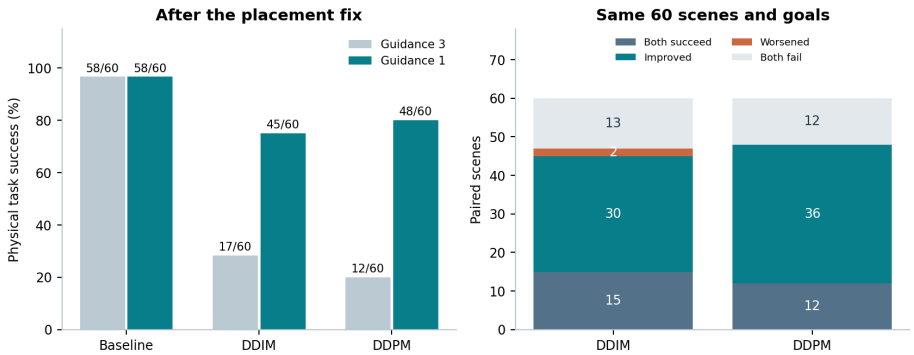
5.2 - Inference step sweep



K	DDIM		DDPM	
	tasks/60	ms/cand.	tasks/60	ms/cand.
10	45	2.89	49	5.40
50	44	13.44	47	23.44
100	**51**	27.30	48	43.41
200	49	49.95	49	75.58
500	49	117.24	44	158.54
1000	49	227.66	**50**	304.55

Earlier pooled setting: 60 scenes, 480 candidates and 60 tasks per configuration, guidance 3, attachment-assisted. The success curve is non-monotonic, so 100 is a practical operating point, not a proven optimum

5.3 - Guidance changed physical performance



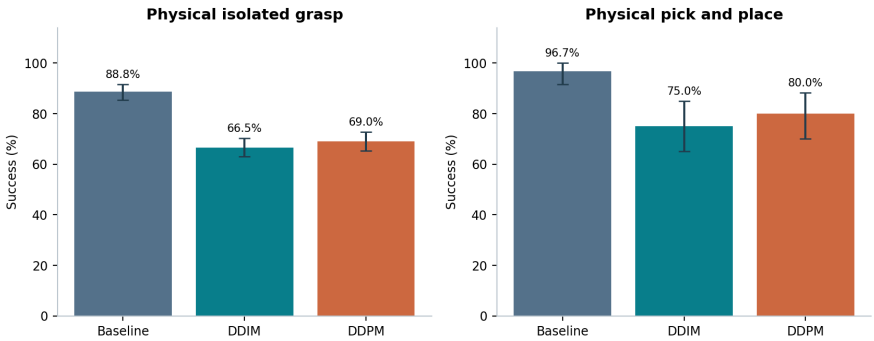
Method	$w = 3$	$w = 1$	Change
DDIM	17/60	45/60	+28
DDPM	12/60	48/60	+36

- Same checkpoint, scenes, goals, step count and physical executor
- Only guidance changed
- Guidance 3 extrapolated the conditional prediction too aggressively for these grasp vectors

Note

The result strongly suggests that guidance 3 was unsuitable for this physical setting. Also, while guidance 1 strongly improves overall physical task success, it isn't necessarily optimal for every object or explain every individual failure.

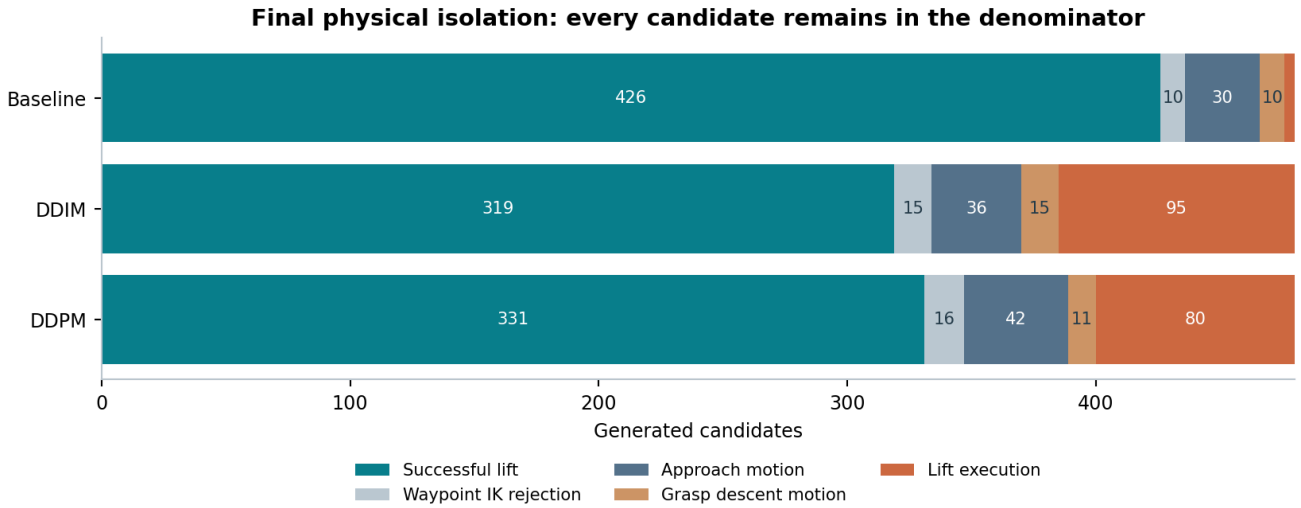
5.4 - Final physical comparison



	Baseline	DDIM 100	DDPM 100
Lifts	426/480	319/480	331/480
Tasks	58/60	45/60	48/60
Task rate	96.67%	75.00%	80.00%
ms/candidate	0.359	30.834	42.638
Pose coverage	100%	95.83%	97.78%

- Baseline remains stronger on these simple cube tasks
- DDPM is slightly higher than DDIM in this cohort, but the evidence does not establish general DDPM superiority
- The learned sampler is functional and physically testable, but the approximation gap remains

5.5 - Failure modes and what they mean



- Geometry/IK success does not guarantee finger contact
- Attachment-assisted runs can hide physical failure
- 6 cm cubes and generated front grasps were weak
- Executor success is not the same as placement accuracy

Key diagnosis

More reverse updates cannot repair biased labels, missing placement conditioning, excessive guidance, or a contact representation that is not physically qualified

6.1 - Conclusion

- We completed a compact conditional diffusion grasp sampler inside TAMPanda
- The model predicts a clean 17-value grasp from a noisy grasp, object condition and time step
- DDIM and DDPM use one frozen model and different reverse schedulers
- A 10 -1,000 step sweep showed higher runtime without monotonic quality improvement, motivating the matched 100-step comparison
- Guidance 1 improved physical task success substantially without retraining
- The analytical baseline remains stronger on the tested cubes

Final conclusion

The method works as a learned proposal source and integrates with the planner, but reliable physical grasping requires better contact-qualified data, richer conditioning and more geometry-aware output representations

Future work: repeat the step sweep under the final physical protocol, condition on placement goals, add varied objects and train on measured contact outcomes